Proposal for

Computer Vision PROJECT
Umm Al Qura University

Project ID: 3

Project Title: Smart Parking detection with proximity recommendations

Team Members:

Student Name	Student ID	Email	Role
Asail Hamdan Al-Osaimi	445000051	s445000051@uqu.edu.sa	Leader
Reema Salem Alharbi	445000219	s445000219@uqu.edu.sa	Member
Enas Abdullah Saud	44512190	s44512190@uqu.edu.sa	Member
Noran Abdullah Aljodi	445005178	s445005178@uqu.edu.sa	Member
Rema Saed Althaqfi	445005754	s445005754@uqu.edu.sa	Member

Proposal for

Computer Vision PROJECT
Umm Al Qura University

Problem statement

Some people struggle to find parking in crowded areas such as universities, markets, and hospitals, leading to wasted time and effort and increased traffic congestion. The lack of real-time information about parking availability limits the use of available spaces and makes the search process ineffective. Therefore, there is a need for an automated system capable of detecting parking conditions and providing real-time information to improve parking management and enhance the user experience.

Motivation and real-world relevance

The system helps reduce time and effort and decreases traffic congestion, while also enhancing the drivers' experience by making the process of finding a parking spot faster and more efficient. It is very useful in real-world environments, especially in crowded places such as universities, hospitals, and malls.

Novelty and expected contribution

The project's novelty lies in the development of an automated, computer vision-based system that detects parking spaces in real time using visual data, rather than relying on manual monitoring or dedicated parking sensors. Unlike traditional parking management solutions, the proposed system employs efficient visual analysis to detect parking availability in congested areas. The novelty stems not from the introduction of a completely new concept, but from the adaptation and implementation of computer vision methods in a practical, scalable manner suitable for real-world settings such as universities, hospitals, and shopping malls.

The expected contribution of this project is to demonstrate how computer vision techniques can improve parking management by accurately identifying vacant and occupied parking spaces. This approach reduces unnecessary vehicle movement, saving drivers' time and effort, minimizing traffic congestion, and highlighting the potential of computer vision as a smart and cost-effective alternative to traditional parking systems.

Proposal for

Computer Vision PROJECT
Umm Al Qura University

Implemented modules/components

Data Preprocessing

The preprocessing stage is responsible for preparing and organizing the parking images before they are passed to the YOLOv8 model for training and inference. Proper preprocessing is essential to ensure dataset consistency, improve detection accuracy, and support efficient model learning.

The dataset utilized in this project is primarily derived from the PKLot parking dataset. To address the time and effort associated with manual image annotation, a pre-labeled version of the same dataset was obtained from Roboflow Universe. This version includes ready-made annotations for parking-related objects and vehicles, represented using bounding boxes in the YOLO format, thereby streamlining the data preparation process and ensuring consistency in labeling.

The preprocessing pipeline includes several important steps:

Dataset Organization:

The dataset is organized into training, validation, and testing subsets to support model training, hyperparameter tuning, and final performance evaluation.

Image Resizing:

All images are automatically resized to 640×640 pixels to ensure compatibility with the YOLOv8 architecture and maintain consistent input dimensions across the dataset.

Image Normalization:

Pixel values are normalized internally by the YOLOv8 framework to improve numerical stability and enhance learning **efficiency during training.**

Label Verification and Preparation:

Bounding box annotations provided in YOLO format are verified to ensure correct object localization and annotation consistency throughout the dataset.

Image Quality Checking:

Images are inspected to ensure readability, proper formatting, and consistency before being processed by the model

Automatic Data Augmentation:

During training, YOLOv8 may automatically apply augmentation techniques such as flipping, scaling, and brightness adjustments to improve model generalization and robustness under different environmental conditions.

The overall objective of preprocessing is to provide clean, standardized, and properly annotated image data that enables accurate and efficient parking occupancy detection using YOLOv8

Proposal for

Computer Vision PROJECT
Umm Al Qura University

Preliminary results

The initial implementation of the system demonstrated successful integration of real-time computer vision for parking space detection and image-based analysis.

High-Precision Detection:

The YOLOv8 model showed strong performance in distinguishing between "empty" and "occupied" parking slots, with accurate localization of available spaces using a confidence threshold of 0.30.

Grid-Based Representation:

The system converts the input image into a structured 120×120 grid to represent parking occupancy in a systematic and organized manner, enabling clear visualization of each parking slot's status.

Real-Time Processing:

The complete pipeline, from image input to output generation, operates efficiently within the Google Colab environment, providing fast and near-instant results.

System Reliability:

Visual evaluation confirmed that the system reliably identifies occupied and free parking areas without overlapping errors, ensuring consistent and accurate output representation

Conclusion:

The preliminary results confirm that the system is fully functional in detecting parking occupancy and presenting structured visual outputs in real time, effectively transitioning from raw image analysis to an organized representation of parking availability

Proposal for

Computer Vision PROJECT
Umm Al Qura University

Code Explanation Step by Step with Screenshots of Outputs

This project consists of two main parts:

The first part focuses on parking space detection and classification using the YOLO model, where parking spaces are identified in images and classified as empty or occupied.

The second part focuses on recommending the nearest available parking space, where the detection results are used to identify and suggest the closest empty spot.

1) Parking space detection and classification Implementation Steps:

First: Dataset Preparation

A parking dataset was downloaded, consisting of real-world images of different parking scenarios, **along with pre-labeled data (Labels)** that specify the state of each parking space within the image. The dataset includes **two main classes: space-empty and space-occupied.**

This dataset serves as the foundation for training the model to learn patterns and accurately distinguish between different parking states.

Second: Model Training using YOLO

The YOLO model was used to train the system to detect and classify parking spaces. No separate preprocessing step was applied before training, as the model relies on its internal processing mechanisms during the training phase.

The training parameters were set as follows:

- **Epochs = 15:** The dataset was passed through the model 15 times, allowing it to gradually learn and improve its performance.
- **Batch Size = 8:** The model was trained on 8 images per iteration, achieving a balance between training speed and memory usage.
- **Image Size = 640:** The input images were set to 640×640 to meet the YOLO model requirements.

During the training process, the model gradually reduces the error value (Loss), which reflects the difference between predicted and actual results. **Through this process, the model learns how to:**

- Detect the location of parking spaces within the image (Object Detection).
- Classify each parking space as either empty or occupied (Classification).

Third: Model Evaluation

After completing the training process, the model was evaluated using the validation dataset. This evaluation aims to measure the model's ability to accurately detect and classify parking spaces.

Several performance metrics were used to assess the model's quality, including:

- **mAP50:** Measures the model's accuracy at an Intersection over Union (IoU) threshold of 0.5
- **mAP50–95:** Provides a more comprehensive evaluation across multiple IoU thresholds
- **Precision:** Measures the proportion of correct positive predictions out of all positive predictions
- **Recall:** Measures the model's ability to detect all relevant instances

High values in these metrics indicate that the model is capable of effectively distinguishing between empty and occupied parking spaces.

Fourth: Model Testing

The model was tested using new data (Test Data) that was not used during training in order to evaluate its ability to generalize to unseen data. The best trained model **(best.pt)** was loaded and used to perform predictions on the test images, with a **confidence threshold of 0.25** applied to ensure that only reliable predictions were considered.

The model analyzed each image, detected the locations of parking spaces, and classified each space as either Empty or Occupied. The prediction results were automatically saved in a designated folder, including images with bounding boxes and labels, for further analysis and presentation.

Fifth: Model Saving

The best trained model **(best.pt)** was saved to Google Drive to ensure it can be reused later without the need for retraining, whether for testing or future use.

Results and Output

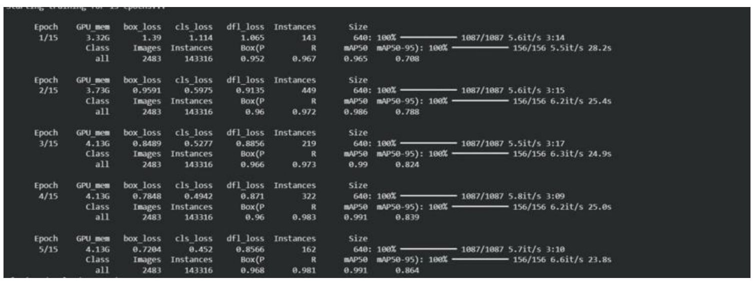
The output results demonstrate the overall performance of the system across the training, evaluation, and testing stages. During training, the model showed a clear improvement over epochs, where the loss values gradually decreased, indicating that the model was learning effectively and improving its detection and classification capabilities.

In the evaluation stage, the model achieved very high-performance metrics, with **mAP50 reaching approximately 0.99** and **mAP50-95 around 0.95**. In addition, the **Precision reached approximately 0.997** and the **Recall reached approximately 0.997**, indicating that the model is highly accurate and capable of correctly detecting and classifying parking spaces.

In the testing stage, the model was applied to unseen images, where it **successfully detected parking spaces and classified them as either empty or occupied**. The output images clearly show bounding boxes around each parking space along with confidence scores, reflecting the reliability of the predictions.

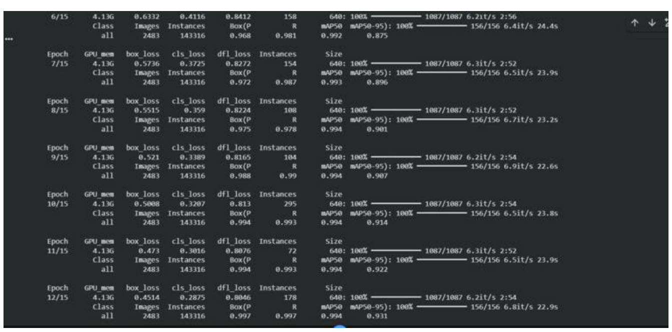
Overall, the results confirm that the model performs effectively across different scenarios and demonstrates strong generalization ability.

Screenshots of the outputs



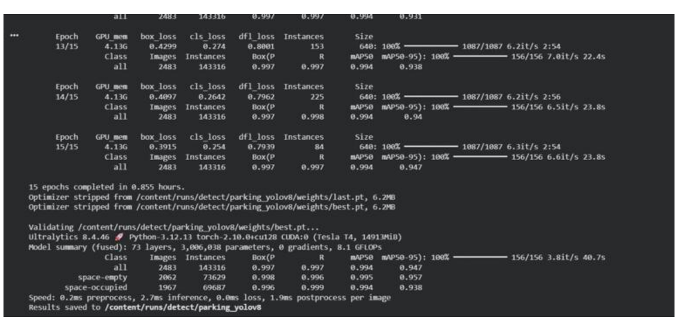
```
Epoch   GPU_mem  box_loss  cls_loss  dfl_loss  Instances    Size
 1/15     3.32G      1.39     1.114     1.065        143     640: 100% ━━━━━━━━ 1087/1087 5.6it/s 3:14
          Class    Images Instances     Box(P          R    mAP50  mAP50-95): 100% ━━━━━━━━ 156/156 5.5it/s 28.2s
            all      2483    143316     0.952      0.967    0.965      0.708

Epoch   GPU_mem  box_loss  cls_loss  dfl_loss  Instances    Size
 2/15     3.73G    0.9591    0.5975    0.9135        449     640: 100% ━━━━━━━━ 1087/1087 5.6it/s 3:15
          Class    Images Instances     Box(P          R    mAP50  mAP50-95): 100% ━━━━━━━━ 156/156 6.2it/s 25.4s
            all      2483    143316      0.96      0.972    0.986      0.788

Epoch   GPU_mem  box_loss  cls_loss  dfl_loss  Instances    Size
 3/15     4.13G    0.8489    0.5277    0.8856        219     640: 100% ━━━━━━━━ 1087/1087 5.5it/s 3:17
          Class    Images Instances     Box(P          R    mAP50  mAP50-95): 100% ━━━━━━━━ 156/156 6.3it/s 24.9s
            all      2483    143316     0.966      0.973     0.99      0.824

Epoch   GPU_mem  box_loss  cls_loss  dfl_loss  Instances    Size
 4/15     4.13G    0.7848    0.4942     0.871        322     640: 100% ━━━━━━━━ 1087/1087 5.8it/s 3:09
          Class    Images Instances     Box(P          R    mAP50  mAP50-95): 100% ━━━━━━━━ 156/156 6.2it/s 25.0s
            all      2483    143316      0.96      0.983    0.991      0.839

Epoch   GPU_mem  box_loss  cls_loss  dfl_loss  Instances    Size
 5/15     4.13G    0.7204     0.452    0.8566        162     640: 100% ━━━━━━━━ 1087/1087 5.7it/s 3:10
          Class    Images Instances     Box(P          R    mAP50  mAP50-95): 100% ━━━━━━━━ 156/156 6.6it/s 23.8s
            all      2483    143316     0.968      0.981    0.991      0.864
```

Figure1: Training progress over epochs showing loss reduction.



```
 6/15     4.13G    0.6332    0.4116    0.8412        158     640: 100% ━━━━━━━━ 1087/1087 6.2it/s 2:56
          Class    Images Instances     Box(P          R    mAP50  mAP50-95): 100% ━━━━━━━━ 156/156 6.4it/s 24.4s
            all      2483    143316     0.968      0.981    0.992      0.875

Epoch   GPU_mem  box_loss  cls_loss  dfl_loss  Instances    Size
 7/15     4.13G    0.5736    0.3725    0.8272        154     640: 100% ━━━━━━━━ 1087/1087 6.3it/s 2:52
          Class    Images Instances     Box(P          R    mAP50  mAP50-95): 100% ━━━━━━━━ 156/156 6.5it/s 23.9s
            all      2483    143316     0.972      0.987    0.993      0.896

Epoch   GPU_mem  box_loss  cls_loss  dfl_loss  Instances    Size
 8/15     4.13G    0.5515     0.359    0.8224        108     640: 100% ━━━━━━━━ 1087/1087 6.3it/s 2:52
          Class    Images Instances     Box(P          R    mAP50  mAP50-95): 100% ━━━━━━━━ 156/156 6.7it/s 23.2s
            all      2483    143316     0.975      0.978    0.994      0.901

Epoch   GPU_mem  box_loss  cls_loss  dfl_loss  Instances    Size
 9/15     4.13G     0.521    0.3389    0.8165        104     640: 100% ━━━━━━━━ 1087/1087 6.2it/s 2:54
          Class    Images Instances     Box(P          R    mAP50  mAP50-95): 100% ━━━━━━━━ 156/156 6.9it/s 22.6s
            all      2483    143316     0.988       0.99    0.994      0.907

Epoch   GPU_mem  box_loss  cls_loss  dfl_loss  Instances    Size
10/15     4.13G    0.5008    0.3207     0.813        295     640: 100% ━━━━━━━━ 1087/1087 6.3it/s 2:54
          Class    Images Instances     Box(P          R    mAP50  mAP50-95): 100% ━━━━━━━━ 156/156 6.5it/s 23.8s
            all      2483    143316     0.994      0.993    0.994      0.914

Epoch   GPU_mem  box_loss  cls_loss  dfl_loss  Instances    Size
11/15     4.13G     0.473    0.3016    0.8076         72     640: 100% ━━━━━━━━ 1087/1087 6.3it/s 2:52
          Class    Images Instances     Box(P          R    mAP50  mAP50-95): 100% ━━━━━━━━ 156/156 6.5it/s 23.9s
            all      2483    143316     0.994      0.993    0.994      0.922

Epoch   GPU_mem  box_loss  cls_loss  dfl_loss  Instances    Size
12/15     4.13G    0.4514    0.2875    0.8046        178     640: 100% ━━━━━━━━ 1087/1087 6.2it/s 2:54
          Class    Images Instances     Box(P          R    mAP50  mAP50-95): 100% ━━━━━━━━ 156/156 6.8it/s 22.9s
            all      2483    143316     0.997      0.997    0.994      0.931
```

Figure2: Continued training results and performance improvement.



```
            all      2483    143316     0.997      0.997    0.994      0.931

Epoch   GPU_mem  box_loss  cls_loss  dfl_loss  Instances    Size
13/15     4.13G    0.4299     0.274    0.8001        153     640: 100% ━━━━━━━━ 1087/1087 6.2it/s 2:54
          Class    Images Instances     Box(P          R    mAP50  mAP50-95): 100% ━━━━━━━━ 156/156 7.0it/s 22.4s
            all      2483    143316     0.997      0.997    0.994      0.938

Epoch   GPU_mem  box_loss  cls_loss  dfl_loss  Instances    Size
14/15     4.13G    0.4097    0.2642    0.7962        225     640: 100% ━━━━━━━━ 1087/1087 6.2it/s 2:56
          Class    Images Instances     Box(P          R    mAP50  mAP50-95): 100% ━━━━━━━━ 156/156 6.5it/s 23.8s
            all      2483    143316     0.997      0.998    0.994       0.94

Epoch   GPU_mem  box_loss  cls_loss  dfl_loss  Instances    Size
15/15     4.13G    0.3915     0.254    0.7939         84     640: 100% ━━━━━━━━ 1087/1087 6.3it/s 2:54
          Class    Images Instances     Box(P          R    mAP50  mAP50-95): 100% ━━━━━━━━ 156/156 6.6it/s 23.8s
            all      2483    143316     0.997      0.997    0.994      0.947

15 epochs completed in 0.855 hours.
Optimizer stripped from /content/runs/detect/parking_yolov8/weights/last.pt, 6.2MB
Optimizer stripped from /content/runs/detect/parking_yolov8/weights/best.pt, 6.2MB

Validating /content/runs/detect/parking_yolov8/weights/best.pt...
Ultralytics 8.4.46 🚀 Python-3.12.13 torch-2.10.0+cu128 CUDA:0 (Tesla T4, 14913MiB)
Model summary (fused): 73 layers, 3,006,038 parameters, 0 gradients, 8.1 GFLOPs
          Class    Images Instances     Box(P          R    mAP50  mAP50-95): 100% ━━━━━━━━ 156/156 3.8it/s 40.7s
            all      2483    143316     0.997      0.997    0.994      0.947
    space-empty      2062     73629     0.998      0.996    0.995      0.957
 space-occupied      1967     69687     0.996      0.999    0.994      0.938
Speed: 0.2ms preprocess, 2.7ms inference, 0.0ms loss, 1.9ms postprocess per image
Results saved to /content/runs/detect/parking_yolov8
```

Figure 3: Final training stage and model performance metrics.

Proposal for

Computer Vision PROJECT
Umm Al Qura University

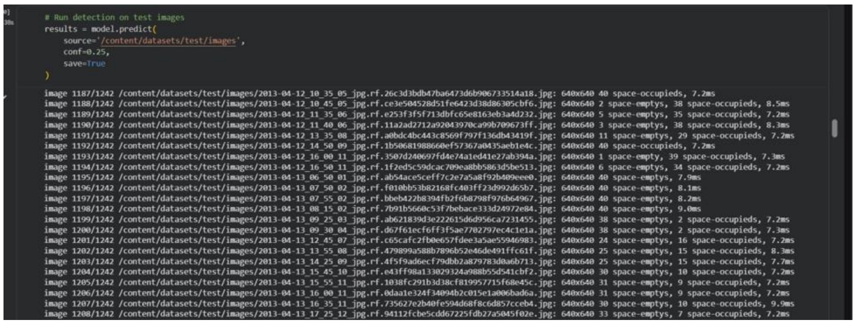


Figure 4: Model prediction results on test images.

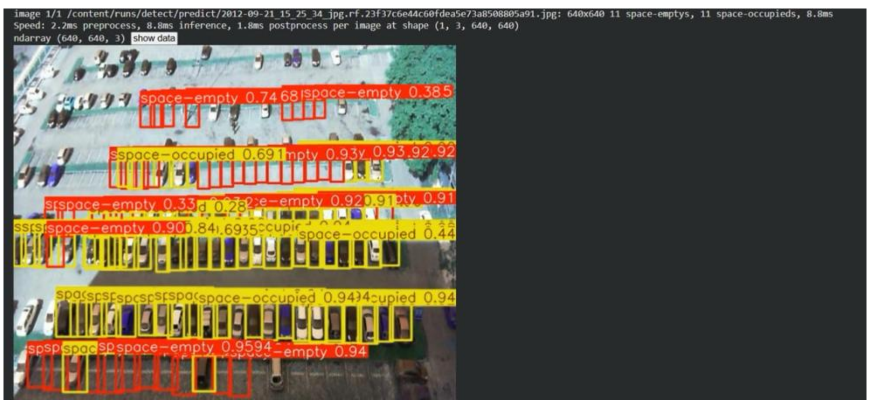


Figure 5: Detection results under different scenarios.

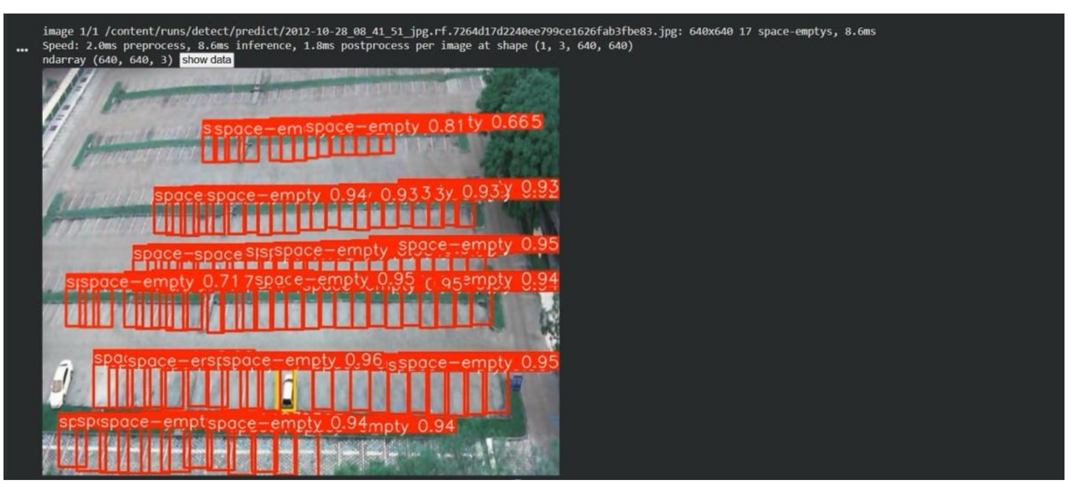


Figure 6: Detection results showing empty parking spaces.

```
image 1/1 /content/runs/detect/predict/2012-10-27_13_15_56_jpg.rf.b97d2c2748c4aad34cd028d9f44c36b8.jpg: 640x640 13 space-emptys, 3 space-occupieds, 9.6ms
Speed: 2.1ms preprocess, 9.6ms inference, 2.0ms postprocess per image at shape (1, 3, 640, 640)
ndarray (640, 640, 3) show data
```

Figure 7: Detection of empty and occupied parking spaces.

```
# Evaluate the trained YOLOv8 model on the validation dataset
metrics = model.val()

# Print evaluation metrics to measure model performance
print("mAP50:", metrics.box.map50)
print("mAP50-95:", metrics.box.map)
print("Precision:", metrics.box.mp)
print("Recall:", metrics.box.mr)

Ultralytics 8.4.46 🚀 Python-3.12.13 torch-2.10.0+cu128 CUDA:0 (Tesla T4, 14913MiB)
Model summary (fused): 73 layers, 3,006,038 parameters, 0 gradients, 8.1 GFLOPs
val: Fast image access ✅ (ping: 0.0±0.0 ms, read: 1700.5±686.6 MB/s, size: 71.7 KB)
val: Scanning /content/datasets/valid/labels.cache... 2483 images, 59 backgrounds, 0 corrupt: 100% ━━━━━━━━━━ 2483/2483 650.9Mit/s 0.0s
                 Class     Images  Instances      Box(P          R      mAP50  mAP50-95): 100% ━━━━━━━━━━ 156/156 3.9it/s 40.1s
                   all       2483     143316      0.997      0.997      0.994      0.951
           space-empty       2062      73629      0.998      0.996      0.995      0.962
        space-occupied       1967      69687      0.996      0.999      0.994      0.941
Speed: 1.2ms preprocess, 3.9ms inference, 0.0ms loss, 1.9ms postprocess per image
Results saved to /content/runs/detect/val
mAP50: 0.9943903997971385
mAP50-95: 0.9512612414038664
Precision: 0.997133264790302
Recall: 0.9971894912922228
```

Figure 8: Evaluation metrics

2) Rrecommending the nearest available parking space Implementation Steps:

In this stage, the results of the parking spot detection model were integrated with the A* pathfinding algorithm to find the shortest route from the user's location to the nearest available parking space. First, the parking image is read and processed using a YOLO model to detect parking spots and classify them as (free / occupied).

The image is then converted into a 2D grid representation to simplify the environment, where some cells represent obstacles and others represent navigable space. The starting point (user location) is defined near the bottom of the image and converted into grid coordinates.

After that, all parking spots classified as free are collected and their centers are mapped into the same grid. The nearest available parking spot is selected based on distance, and the A* algorithm is applied to compute the shortest path between the user and the selected spot.

Finally, the computed path is visualized on the image using plotted points that illustrate the route, along with marking the user's position and the chosen parking space.

Screenshots of the outputs

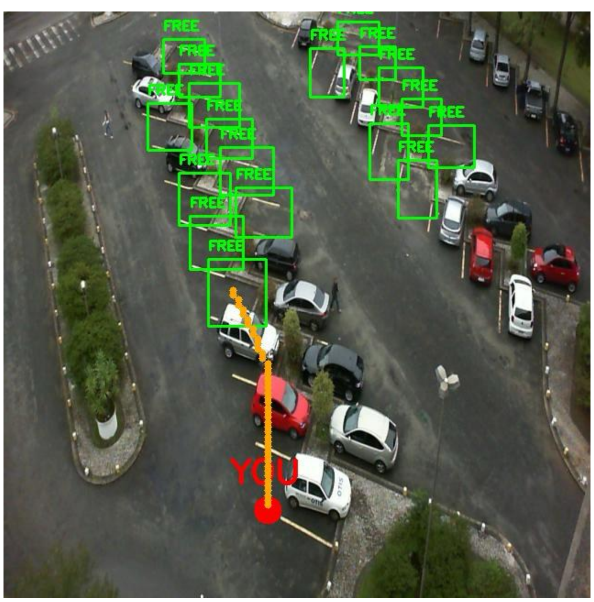


Figure 9: Shortest path to the nearest available parking space.

Proposal for

Computer Vision PROJECT
Umm Al Qura University

جـامعـــة أم القـــرى
UMM AL-QURA UNIVERSITY

Technical Challenges and Solutions

1. Total Failure of Traditional Detection Methods (HOG)

- **Challenge**: Initial attempts to implement the **HOG (Histogram of Oriented Gradients)** algorithm yielded no functional results. The algorithm failed to produce any detections or meaningful output when applied to the parking dataset, proving it completely incapable of handling the complexity of the task.
- **Solution**: Recognizing this total failure, the project immediately pivoted to a **Deep Learning** approach using **YOLOv8**. This shift was essential as it moved from a failing manual feature extraction method to a robust, automated neural network capable of processing the data successfully.

2. Hardware Constraints and Computational Demands

- **Challenge**: Training advanced computer vision models like YOLOv8 imposed a significant burden on the available hardware. The initial device lacked the necessary **GPU capabilities**, leading to excessive training times and extreme latency that hindered the development progress.
- **Solution**: To overcome this infrastructure bottleneck, the workload was migrated to a high-performance laptop equipped with an **NVIDIA GPU**. By leveraging **CUDA-accelerated** parallel processing, the training cycle was optimized.

3. Software Library Compatibility

- **Challenge**: Security updates in the software libraries introduced restrictive protocols that blocked the model from loading its pre-trained weights.
- **Solution**: This was resolved by implementing a **programmatic override**. A custom function wrapper was used to adjust the loading parameters, ensuring a seamless restoration of the model without disrupting the environment stability.

4. Optimization of Architectural Efficiency

- **Challenge**: Identifying a model configuration that delivers high precision while remaining computationally efficient for processing data.
- **Solution**: The **YOLOv8 Nano** variant was selected for its optimized architecture. It achieved a remarkable **mAP50 of 99.4%**, successfully balancing high precision with lightweight resource consumption.

Proposal for

Computer Vision PROJECT
Umm Al Qura University

• Updated task distribution among team members

Student Name	Initial task
Enas Abdullah Saud	This component was collaboratively developed to implement the A* algorithm for optimal pathfinding to the nearest available parking space.
Reema Salem Alharbi	Data Preparation, Labeling, and Preliminary Results Analysis
Rema Saed Althaqfi	This component was collaboratively developed to implement the YOLO model for parking space detection and classification, where parking spaces were detected and classified as either empty or occupied.
Noran Abdullah Aljodi	This component was collaboratively developed to implement the YOLO model for parking space detection and classification, where parking spaces were detected and classified as either empty or occupied.
Asail Hamdan Al-Osaimi	This component was collaboratively developed to implement the A* algorithm for optimal pathfinding to the nearest available parking space.

References

[1] P. Almeida, L. S. Oliveira, E. Silva Jr., A. Britto Jr., and A. Koerich, "PKLot – A robust dataset for parking lot classification," Expert Systems with Applications, vol. 42, no. 11, pp. 4937–4949, 2015.

[2] A. M. N. Alhajali, "PKLot Dataset," Kaggle, 2023. [Online]. Available: https://www.kaggle.com/datasets/ammarnassanalhajali/pklot-dataset. [Accessed: Mar. 4, 2026].